

Assignment 4: DataFrameLib

Build a High-Performance DataFrame Library in C++

COP290 – Design Practices

Group Size: This assignment must be done **individually**.

Deadline: 27 April 2026, 11:59 PM.

Contents

1	Background	2
2	Project Description	2
3	Required Operations	3
3.1	I/O Functions	3
3.2	DataFrame Operations	3
3.3	Expression System	3
3.3.1	Core Expressions	3
3.3.2	Arithmetic Operations	3
3.3.3	Comparison and Boolean Operations	3
3.3.4	String Functions	4
3.3.5	Aggregation Functions	4
4	Lazy Evaluation	4
5	Query Optimization	4
6	Example Usage	5
6.1	Eager Mode	5
6.2	Lazy Mode	5
7	Evaluation Rubric	5
7.1	Evaluation Criteria	5
7.2	Evaluation Method	5
8	Submission Instructions	5

1 Background

Tabular data processing is a core task in data engineering and data science. Libraries such as **Pandas** (Python) and **Polars** (Rust/Python) have become standard tools for working with structured, column-oriented datasets. Pandas provides an intuitive, *eager* execution model — every operation runs immediately and returns a materialised result. Polars, on the other hand, exposes a *lazy* API that builds a computation plan first and then optimises and executes it on demand, enabling significant performance gains through query optimisation.

In this assignment you will build **DataFrameLib**, a C++ library that replicates the core ideas behind both paradigms: an eager execution path for immediate, interactive use, and a lazy execution path backed by a query optimiser. We recommend getting familiar with Polars and Pandas if you have not used them before.

2 Project Description

Build **DataFrameLib**, a C++ library for tabular data processing with two execution modes and a query optimiser:

- **EagerDataFrame** — immediate execution with materialised data.
- **LazyDataFrame** — deferred execution via a computation DAG.
- **QueryOptimizer** — optimises lazy execution plans before materialisation.

Apache Arrow

Use Apache Arrow exclusively for all data storage and I/O:

- In-memory columnar data representation.
- Reading and writing CSV and Parquet files.
- Type-safe column operations.

Why Arrow? Language-independent format, zero-copy sharing, cache-friendly columnar layout, and built-in I/O support. Documentation: <https://arrow.apache.org/docs/cpp/>

Type Safety

The library must enforce strict type safety:

- Each column has an immutable type: `int32`, `int64`, `float32`, `float64`, `string`, or `boolean`.
- Missing values are represented as `null` only — not as NaN.
- Type promotion rule: `int + float = float`.
- Incompatible operations (e.g. adding a numeric and a non-numeric column) must throw errors immediately. Any operation involving a `null` operand must produce a `null` result.

3 Required Operations

3.1 I/O Functions

Function	Description
<code>read_csv(path)</code>	Load CSV into an EagerDataFrame
<code>read_parquet(path)</code>	Load Parquet into an EagerDataFrame
<code>scan_csv(path)</code>	Create a LazyDataFrame from CSV
<code>scan_parquet(path)</code>	Create a LazyDataFrame from Parquet
<code>write_csv(path)</code>	Write an EagerDataFrame to CSV
<code>write_parquet(path)</code>	Write an EagerDataFrame to Parquet
<code>sink_csv(path)</code>	Write a LazyDataFrame result to CSV
<code>sink_parquet(path)</code>	Write a LazyDataFrame result to Parquet
<code>from_columns(map)</code>	Build a DataFrame from a column map

3.2 DataFrame Operations

Both EagerDataFrame and LazyDataFrame must support:

Operation	Description	Example
<code>select(columns)</code>	Select columns/expressions	<code>df.select({"a", "b"})</code>
<code>filter(predicate)</code>	Filter rows by a predicate	<code>df.filter(col("x") > 0)</code>
<code>with_column(name, expr)</code>	Add or replace a column	<code>df.with_column("z", col("a")+col("b"))</code>
<code>group_by(keys)</code>	Group by one or more columns	<code>df.group_by({"dept"})</code>
<code>aggregate(agg_map)</code>	Aggregate grouped data	<code>df.aggregate({"x": "sum"})</code>
<code>join(other, on, how)</code>	Join two DataFrames	<code>df.join(df2, {"id"}, "inner")</code>
<code>sort(columns, asc)</code>	Sort rows	<code>df.sort({"age"}, true)</code>
<code>head(n)</code>	First <i>n</i> rows	<code>df.head(10)</code>
<code>collect()</code>	<i>[Lazy only]</i> Execute and materialise	<code>df.collect()</code>

3.3 Expression System

3.3.1 Core Expressions

Expression	Description	Example
<code>col(name)</code>	Column reference	<code>col("age")</code>
<code>lit(value)</code>	Literal value	<code>lit(100)</code>
<code>alias(name)</code>	Rename result	<code>col("x").alias("y")</code>

3.3.2 Arithmetic Operations

Operation	Description	Example
<code>+ - * /</code>	Element-wise arithmetic	<code>col("x") * col("y")</code>
<code>%</code>	Modulo	<code>col("id") % 10</code>
<code>abs()</code>	Absolute value	<code>col("x").abs()</code>

3.3.3 Comparison and Boolean Operations

Operation	Description	Example
<code>== != < <= > >=</code>	Comparisons	<code>col("age") > 30</code>
<code>& ~</code>	Boolean AND, OR, NOT	<code>(a > 0) & (b < 5)</code>
<code>is_null()</code>	Check for nulls	<code>col("x").is_null()</code>
<code>is_not_null()</code>	Check for non-nulls	<code>col("x").is_not_null()</code>

3.3.4 String Functions

Function	Description	Example
<code>length()</code>	String length	<code>col("name").length()</code>
<code>contains(s)</code>	Substring check	<code>col("email").contains("@")</code>
<code>starts_with(s)</code>	Prefix check	<code>col("code").starts_with("A")</code>
<code>ends_with(s)</code>	Suffix check	<code>col("file").ends_with(".txt")</code>
<code>to_lower()</code>	Lowercase	<code>col("city").to_lower()</code>
<code>to_upper()</code>	Uppercase	<code>col("city").to_upper()</code>

3.3.5 Aggregation Functions

Function	Description	Example
<code>sum()</code>	Sum of values	<code>col("salary").sum()</code>
<code>mean()</code>	Mean value	<code>col("score").mean()</code>
<code>count()</code>	Count of non-null values	<code>col("id").count()</code>
<code>min()</code>	Minimum value	<code>col("age").min()</code>
<code>max()</code>	Maximum value	<code>col("age").max()</code>

4 Lazy Evaluation

`LazyDataFrame` builds a directed acyclic graph (DAG) where:

- **Nodes** — operations (scan, filter, select, join, ...)
- **Edges** — data dependencies between operations
- **Leaf nodes** — data sources (CSV / Parquet scans)

No data is processed during plan construction; execution is triggered only by `collect()`. You must also implement a graph-rendering utility — using Graphviz or `Boost.Graph` — that dumps the computation DAG to a `.png` file when `explain(path)` is called.

5 Query Optimization

For the same query, different execution plans can be generated, giving rise to optimisation opportunities. For example, joining two large DataFrames and then applying a filter is far more expensive than filtering first and then joining, even though the final output is identical. You must implement rule-based transformations that preserve correctness.

Part of the grading in this section is based on the running time of a fixed set of query benchmarks (i.e, the speedups the query optimizer introduces). A baseline is established for unoptimised plans; any submission that performs worse than the baseline receives zero for this section.

Some standard optimisations are listed below. You are free to explore and implement additional ones.

Optimisation	Description
Predicate Pushdown	Move filter operations closer to scan nodes
Projection Pushdown	Read only the columns required by downstream operations
Constant Folding	Evaluate constant expressions at plan-construction time
Expression Simplification	Simplify trivial expressions, e.g. $x*1+0 \rightarrow x$
Limit Pushdown	Apply row-count limits as early as possible

For each optimisation you implement, your report must: describe the transformation, prove its correctness, provide a concrete example, and explain the expected performance benefit.

6 Example Usage

6.1 Eager Mode

```
1 auto df = read_parquet("data.parquet");
2 auto result = df.filter(col("age") > 30)
3                 .select({"name", "salary"})
4                 .write_csv("output.csv");
```

6.2 Lazy Mode

```
1 auto df = scan_parquet("data.parquet");
2 auto result = df
3     .filter(col("age") > 30)
4     .select({"name", "salary"})
5     .group_by({"dept"})
6     .aggregate({"avg_sal", col("salary").mean()});
7
8 std::string plan_path = "plan.png";
9 result.explain(plan_path); // Dump the optimised DAG
10 auto output = result.collect(); // Execute
```

7 Evaluation Rubric

7.1 Evaluation Criteria

Component	Weight	Description
Correctness	45%	Autograded against TA-designed test cases covering all required operations.
Query Optimisation	15%	Graded by benchmark running time relative to other submissions. Performance below the unoptimised baseline receives zero.
Memory Management & Performance	15 %	Assessed for proper use of RAII, smart pointers, move semantics, and absence of memory leaks.
Code Quality & Documentation	10%	Clarity, organisation, and standard practices. The public API must be thoroughly documented.
Viva	15%	

Note: A comprehensive list of tested operations and a set of sample test cases will be shared shortly.

7.2 Evaluation Method

Correctness is evaluated through TA-designed test cases that exercise the library via its public API. Please follow the API signatures in this document exactly — any deviation may cause tests to fail. There will additionally be a demo and viva; if the implementation is correct but viva performance falls short of expectations, marks may be significantly reduced.

8 Submission Instructions

1. You must submit:

- Complete source code implementing `EagerDataFrame`, `LazyDataFrame`, and `QueryOptimizer`.
- Build and installation instructions (`README`).

- A design document (**report.pdf**) describing your architecture and all optimisations implemented.
2. The project must compile using standard CMake. **Do not** modify the build system or introduce external libraries beyond Apache Arrow and Graphviz / Boost.Graph.
 3. Before packaging, remove all compiled binaries and build artefacts (e.g. delete the **build/** directory). Submissions containing unnecessary build files may receive a penalty.
 4. Do not include generated output directories or data files in your archive.
 5. We will use modern AI-based plagiarism checkers. A match will result in **zero** for this assignment and another assignment of our choosing.
 6. There may additionally be a demo and viva. It is possible that the assignment works correctly but viva performance falls short of expectations, in which case marks may be significantly reduced.
 7. All submissions will be via **Moodle**. Create a tar file as follows:
 - (a) Place all your files in a directory named **project**.
 - (b) Navigate to the parent directory of **project**.
 - (c) Run: `tar -cvf <entry_number>.tar project`
 - (d) An entry number is of the form 2025ANZ8223 — it is **not** your user ID or email address.
 - (e) Submit the resulting tar file on Moodle.
 8. Failure to follow the required submission format may result in a penalty.
 9. Submit several hours before the deadline and double-check your submission.
 10. **No emails will be entertained. Kindly use Piazza for all discussions, clarifications and announcements.**